

Task 1

ER Diagram - Electronic Voting System



Submitted By : Lokesh Bhatta

Email : blokesh24@tbc.edu.np

UWE ID : 24071110

Section : A

Submitted To : Mr Rabin Gautam

The British College Kathmandu

Date : April 26, 2026

Introduction

This report presents the conceptual design of an Electronic Voting System along with its Entity-Relationship Diagram(ERD), relational schema diagram and lastly its entire SQL schema. The ERD is modeled using Chen notation to clearly represent entities, attributes, relationships, cardinalities, and constraints. The system aims to ensure secure, transparent, and accurate handling of voters, registrations, elections, candidates, votes, and results.

The entire project is also uploaded in my Github. The link to the github repo is here:

<https://github.com/LokeshBhatta2428/Electronic-Voting-System.git>

Entity-Relationship Diagram (ERD)

The ER Diagram below illustrates the conceptual entities and their relationships. Key features include the ISA relationship between Voters and their registration status, and the linkage between candidates and political parties.

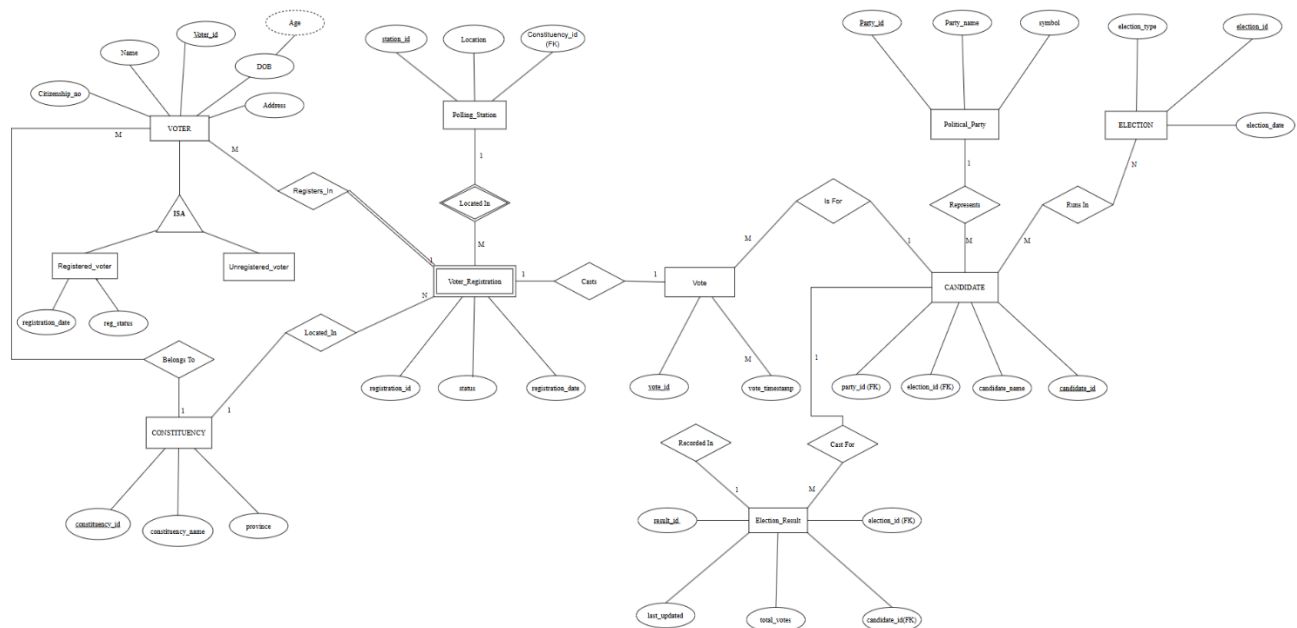


fig-1: Entity-Relationship Diagram

This ERD was made using <https://app.diagrams.net/>

The Google drive link to this diagram is here :

https://drive.google.com/file/d/1_dDmUrHv_KdAoLHo0_RPjbf_S1fftZlC/view?usp=sharing

make sure to open with **draw.io** for efficiency.

Relational Schema

Converting the conceptual ERD into a logical relational schema results in the following tables.

Primary keys are indicated as (PK) and foreign keys are indicated as (FK), to maintain referential integrity.

- **constituency** (constituency_id (PK), constituency_name, province)
- **polling_station** (station_id (PK), location, constituency_id (FK))
- **voter** (voter_id (PK), name, citizenship_no, address, dob, constituency_id (FK))
- **registered_voter** (voter_id(PK, FK), registration_date, reg_status)
- **unregistered_voter** (voter_id (PK, FK))
- **voter_registration** (registration_id (PK), voter_id(FK), station_id (FK), constituency_id(FK))
- **political_party** (party_id(PK), party_name, symbol)
- **election** (election_id(PK), election_type, election_date)
- **candidate** (candidate_id (PK), candidate_name, party_id (FK), election_id (FK))
- **vote** (vote_id (PK), registration_id(FK), candidate_id (FK), election_id(FK), vote_timestamp)
- **election_result** (result_id(PK), election_id(FK), candidate_id(FK), total_votes, result_status)

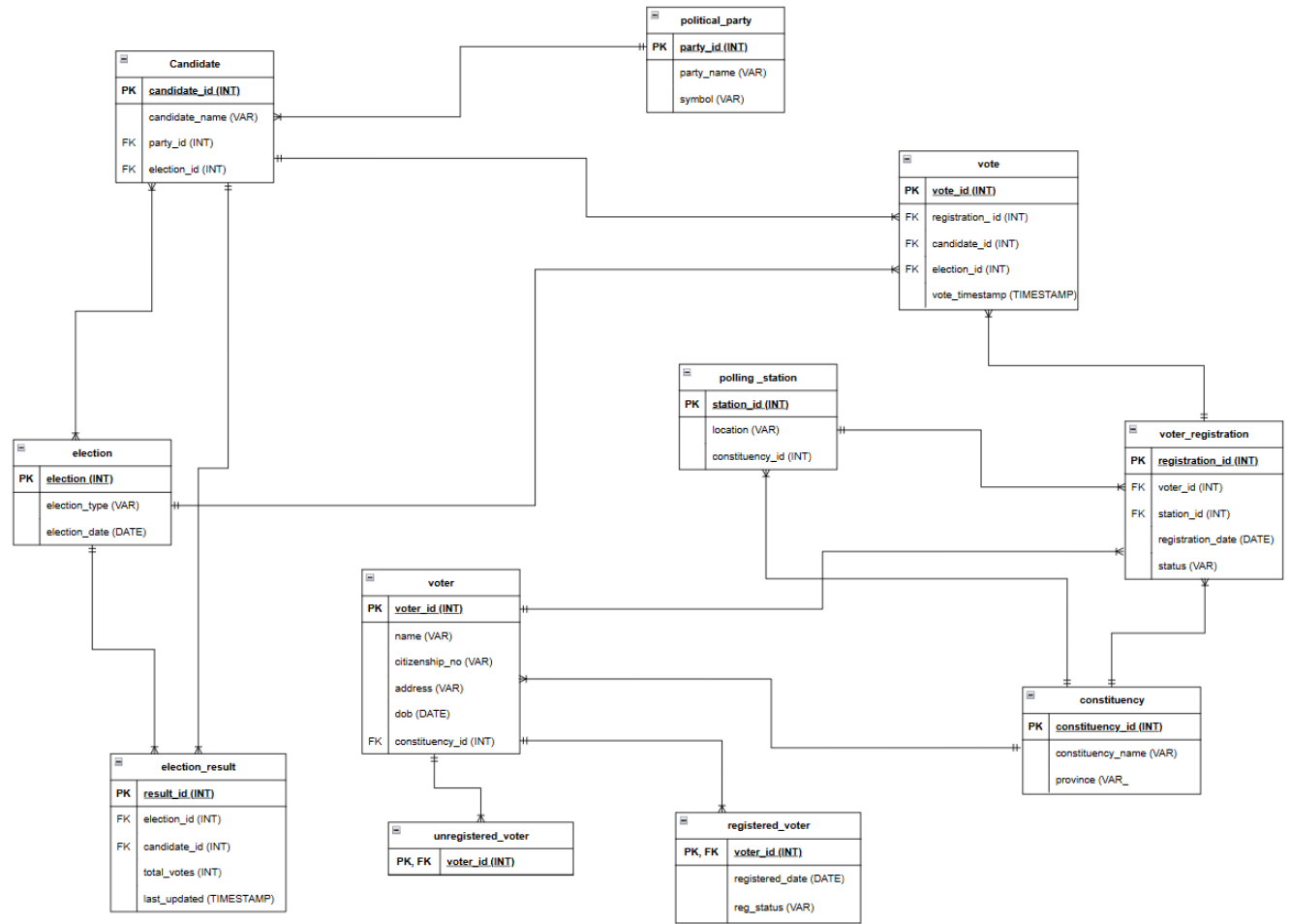


fig-2: Relational Schema Diagram

This relational schema diagram was made using <https://app.diagrams.net/>.

The Google drive link to this diagram is here :

https://drive.google.com/file/d/17vGvafwiwY58TIEsUCbBfPB_7obnrgSh/view?usp=sharing
make sure to open with **draw.io** for efficiency.

SQL Script

The following SQL script outlines the physical implementation of the database, including table structures and foreign key constraints.

```
Create Database Election;  
use election;
```

```
CREATE TABLE constituency (  
    constituency_id INT PRIMARY KEY,  
    constituency_name VARCHAR(150),  
    province VARCHAR(100)  
);
```

```
CREATE TABLE polling_station (  
    station_id INT PRIMARY KEY,  
    location VARCHAR(255),  
    constituency_id INT,  
    FOREIGN KEY (constituency_id) REFERENCES constituency(constituency_id)  
);
```

```
CREATE TABLE voter (  
    voter_id INT PRIMARY KEY,  
    name VARCHAR(150),  
    citizenship_no VARCHAR(30),  
    address VARCHAR(255),  
    dob DATE,  
    constituency_id INT,  
    FOREIGN KEY (constituency_id) REFERENCES constituency(constituency_id)  
);
```

```
CREATE TABLE registered_voter (  
    voter_id INT PRIMARY KEY,  
    registration_date DATE,  
    reg_status VARCHAR(20),  
    FOREIGN KEY (voter_id) REFERENCES voter(voter_id)  
);
```

```
CREATE TABLE unregistered_voter (  
    voter_id INT PRIMARY KEY,  
    FOREIGN KEY (voter_id) REFERENCES voter(voter_id)  
);
```

```
CREATE TABLE voter_registration (  
    voter_id INT PRIMARY KEY,  
    registration_date DATE,  
    reg_status VARCHAR(20),  
    FOREIGN KEY (voter_id) REFERENCES voter(voter_id)  
);
```

```

registration_id INT      PRIMARY KEY,
voter_id      INT,
station_id    INT,
constituency_id INT,
registration_date DATE,
status        VARCHAR(20),
FOREIGN KEY (voter_id) REFERENCES voter(voter_id),
FOREIGN KEY (station_id) REFERENCES polling_station(station_id),
FOREIGN KEY (constituency_id) REFERENCES constituency(constituency_id)
);

CREATE TABLE political_party (
    party_id      INT      PRIMARY KEY,
    party_name    VARCHAR(150),
    symbol        VARCHAR(100)
);

CREATE TABLE election (
    election_id    INT      PRIMARY KEY,
    election_type  VARCHAR(50),
    election_date  DATE
);

CREATE TABLE candidate (
    candidate_id   INT      PRIMARY KEY,
    candidate_name VARCHAR(150),
    party_id       INT,
    election_id    INT,
    FOREIGN KEY (party_id) REFERENCES political_party(party_id),
    FOREIGN KEY (election_id) REFERENCES election(election_id)
);

CREATE TABLE vote (
    vote_id        INT      PRIMARY KEY,
    registration_id INT,
    candidate_id    INT,
    election_id     INT,
    vote_timestamp  TIMESTAMP,
    FOREIGN KEY (registration_id) REFERENCES voter_registration(registration_id),
    FOREIGN KEY (candidate_id) REFERENCES candidate(candidate_id),
    FOREIGN KEY (election_id) REFERENCES election(election_id)
);

CREATE TABLE election_result (
    result_id      INT      PRIMARY KEY,
    election_id     INT,

```

```
candidate_id    INT,  
total_votes     INT,  
last_updated    TIMESTAMP,  
FOREIGN KEY (election_id) REFERENCES election(election_id),  
FOREIGN KEY (candidate_id) REFERENCES candidate(candidate_id)  
);
```